# HowAreYou GH

## Frontend Developer Documentation

For: Carey | Stack: Next.js + TailwindCSS | Cursor Ghana Hackathon @ KNUST

**Your mission: build every screen the user sees.**

You own the web app (Next.js), the design system, the API integration, the counsellor admin view, and the pitch demo. This document tells you exactly what to build, screen by screen, component by component.

### SECTIONS IN THIS DOCUMENT

# 01 Project Setup & Folder Structure

### Step 1 — Install Node.js

Check you have Node 18+:

```
node --version
```

Download from nodejs.org if needed.

### Step 2 — Create the Next.js project

```
npx create-next-app@latest wellcheck-web

# Answer the prompts like this:
# TypeScript?              Yes
# ESLint?                  Yes
# Tailwind CSS?            Yes
# src/ directory?          Yes
# App Router?              Yes
# Import alias (@/*)?      Yes
```

### Step 3 — Install additional dependencies

```
cd wellcheck-web
npm install @supabase/supabase-js lucide-react recharts
```

| Package | What it does |
|---|---|
| @supabase/supabase-js | Connects to your Supabase database and handles auth |
| lucide-react | Clean, free icon library — mic, alert, heart, history icons etc. |
| recharts | For the mood history chart on the history screen |

### Step 4 — Environment variables

Create a .env.local file in the root of the project:

```
NEXT_PUBLIC_SUPABASE_URL=https://your-project.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=your-anon-key-here
NEXT_PUBLIC_API_BASE_URL=http://localhost:8000
# Replace the API_BASE_URL with Darktrace's Railway URL once deployed
```

ⓘ Variables prefixed with NEXT_PUBLIC_ are accessible in the browser. Variables without it are server-only. Both the Supabase URL and the API base URL are safe to be public.

### Step 5 — Folder structure

```
wellcheck-web/
├── src/
│   ├── app/                      ← Next.js App Router pages
│   │   ├── layout.tsx            ← root layout (fonts, global styles)
│   │   ├── page.tsx              ← redirects to /checkin if logged in
│   │   ├── login/page.tsx        ← login + signup screen
│   │   ├── checkin/page.tsx      ← main check-in screen
│   │   ├── result/page.tsx       ← result screen (normal + red alert)
│   │   ├── history/page.tsx      ← mood history screen
│   │   └── admin/page.tsx        ← counsellor admin view
│   ├── components/               ← reusable UI components
│   │   ├── UrgencyBadge.tsx
│   │   ├── ResultCard.tsx
│   │   ├── MicButton.tsx
│   │   ├── LanguageSelector.tsx
│   │   ├── LoadingSkeleton.tsx
│   │   └── CounsellorCard.tsx
│   ├── lib/
│   │   ├── api.ts                ← all calls to Darktrace's FastAPI
│   │   ├── supabase.ts           ← Supabase client setup
│   │   └── useAuth.ts            ← auth hook (get current user)
│   └── styles/
│       └── globals.css           ← Tailwind base styles + custom CSS
├── .env.local                    ← secret keys (never commit)
├── .gitignore
└── tailwind.config.ts
```

### Step 6 — Configure Tailwind with your design tokens

Open tailwind.config.ts and add your custom colours so you can use them as Tailwind classes throughout the app:

```
import type { Config } from 'tailwindcss'

const config: Config = {
  content: ['./src/**/*.{js,ts,jsx,tsx}'],
  theme: {
    extend: {
      colors: {
        primary:       '#1A56DB',
        'primary-light': '#EEF2FF',
        dark:          '#1E1E2E',
        surface:       '#F9FAFB',
        'urgency-low':    '#16A34A',
        'urgency-low-bg': '#F0FDF4',
```

```
        'urgency-mid':    '#D97706',
        'urgency-mid-bg': '#FFFBEB',
        'urgency-high':   '#DC2626',
        'urgency-high-bg':'#FEF2F2',
        wellness:      '#7C3AED',
        'wellness-bg': '#F5F3FF',
      },
      fontFamily: {
        sans: ['Inter', 'system-ui', 'sans-serif'],
      },
    },
  },
  plugins: [],
}

export default config
```

## 02 Design Tokens — Colours, Typography, Spacing

These are the rules that govern every visual decision in the app. Follow them consistently across every screen.

### Colour system

| Token | Hex — When to use |
|---|---|
| primary (#1A56DB) | Buttons, active states, links, focus rings, key icons |
| primary-light (#EEF2FF) | Card backgrounds, info banners, selected states |
| dark (#1E1E2E) | All body text, headings — never pure black |
| surface (#F9FAFB) | Page background — warm off-white, never pure white |
| urgency-low (#16A34A) | LOW badge text and icon only |
| urgency-low-bg (#F0FDF4) | LOW card background |
| urgency-mid (#D97706) | MEDIUM badge text and icon only |
| urgency-mid-bg (#FFFBEB) | MEDIUM card background |
| urgency-high (#DC2626) | HIGH badge, red alert screen — NOWHERE ELSE |
| urgency-high-bg (#FEF2F2) | HIGH card background (non-crisis) |
| wellness (#7C3AED) | Mental wellness section accent |
| wellness-bg (#F5F3FF) | Mental wellness card background |

NON-NEGOTIABLE: Red (#DC2626) must NEVER appear anywhere except HIGH urgency. Not for errors, not for delete buttons, not for required fields. If red appears casually elsewhere, it loses all impact when it actually matters.

### Typography scale

| Use case | Size / Weight / Class |
|---|---|
| App name / hero headline | text-3xl font-bold (30px) |
| Screen title | text-2xl font-bold (24px) |
| Section heading / card title | text-lg font-semibold (18px) |
| Body text / result content | text-base (16px) — default |

| | |
|---|---|
| Secondary / caption / label | text-sm (14px) |
| Badge / tag text | text-xs font-bold uppercase tracking-wide (12px) |

ⓘ Add Inter from Google Fonts. In src/app/layout.tsx, import it via next/font/google. This is the standard Next.js way — no external link tags needed.

## Spacing rules (8-point grid)

All spacing values must be multiples of 8. Use Tailwind spacing classes — they map directly:

| Tailwind class | Value — Use for |
|---|---|
| p-2 / gap-2 | 8px — tight internal padding |
| p-4 / gap-4 | 16px — card internal padding (minimum) |
| p-6 / gap-6 | 24px — section padding, comfortable card padding |
| p-8 / gap-8 | 32px — generous section spacing |
| p-12 | 48px — hero sections, top of page padding |
| rounded-xl | 12px radius — all cards |
| rounded-2xl | 16px radius — large cards, modals |
| rounded-full | Full radius — badges, mic button, avatars |

## 03 Reusable Components Library

Build these components first in src/components/ before building any screens. Every screen uses them.

### UrgencyBadge.tsx

Shows LOW / MEDIUM / HIGH as a coloured pill badge. Used on the result screen and the admin view.

```
// src/components/UrgencyBadge.tsx
type Props = { level: 'LOW' | 'MEDIUM' | 'HIGH' | null }

const config = {
  LOW:    { label: 'LOW',    bg: 'bg-urgency-low-bg',  text: 'text-urgency-low',
icon: '✓' },
  MEDIUM: { label: 'MEDIUM', bg: 'bg-urgency-mid-bg',  text: 'text-urgency-mid',
icon: '!' },
  HIGH:   { label: 'HIGH',   bg: 'bg-urgency-high-bg', text: 'text-urgency-high',
icon: '⚠' },
}

export default function UrgencyBadge({ level }: Props) {
  if (!level) return null
  const c = config[level]
  return (
    <span className={`inline-flex items-center gap-1 px-3 py-1 rounded-full
      text-xs font-bold uppercase tracking-wide ${c.bg} ${c.text}`}>
      <span>{c.icon}</span>
      <span>{c.label}</span>
    </span>
  )
}
```

### ResultCard.tsx

A card container used to display each section of the result (causes, urgency, advice, wellness response). Accepts a title, icon, background colour, and children.

```
// src/components/ResultCard.tsx
type Props = {
  title: string
  icon?: string
  bgClass?: string   // e.g. 'bg-primary-light' or 'bg-wellness-bg'
  children: React.ReactNode
}

export default function ResultCard({ title, icon, bgClass = 'bg-white', children
}: Props) {
```

```
  return (
    <div className={`${bgClass} rounded-xl p-6 border border-gray-100 shadow-sm`}>
      <h3 className='text-lg font-semibold text-dark mb-3'>
        {icon && <span className='mr-2'>{icon}</span>}{title}
      </h3>
      {children}
    </div>
  )
}
```

### MicButton.tsx

The circular mic button for voice input. Shows idle, recording, and processing states. Pulses gently when recording.

```
// src/components/MicButton.tsx
'use client'
import { Mic, MicOff, Loader } from 'lucide-react'

type State = 'idle' | 'recording' | 'processing'
type Props = { state: State; onPress: () => void }

export default function MicButton({ state, onPress }: Props) {
  const base = 'w-16 h-16 rounded-full flex items-center justify-center
transition-all'
  const styles = {
    idle:       `${base} bg-primary text-white hover:bg-primary/90 shadow-lg`,
    recording:  `${base} bg-red-500 text-white animate-pulse shadow-xl scale-110`,
    processing: `${base} bg-gray-200 text-gray-400 cursor-not-allowed`,
  }
  return (
    <button onClick={onPress} disabled={state === 'processing'}
      className={styles[state]} aria-label='Voice input'>
      {state === 'idle'       && <Mic size={28} />}
      {state === 'recording'  && <MicOff size={28} />}
      {state === 'processing' && <Loader size={28} className='animate-spin' />}
    </button>
  )
}
```

### LanguageSelector.tsx

A pill-style toggle to switch between English, Twi, Ga, and Ewe. When anything other than English is selected, the mic button appears and the text input shows a note.

```
// src/components/LanguageSelector.tsx
'use client'

const LANGUAGES = [
  { code: 'en',  label: 'English' },
```

```
  { code: 'tw',  label: 'Twi' },
  { code: 'ga',  label: 'Ga' },
  { code: 'ee',  label: 'Ewe' },
]

type Props = { selected: string; onChange: (code: string) => void }

export default function LanguageSelector({ selected, onChange }: Props) {
  return (
    <div className='flex gap-2 flex-wrap'>
      {LANGUAGES.map(lang => (
        <button key={lang.code}
          onClick={() => onChange(lang.code)}
          className={`px-4 py-1.5 rounded-full text-sm font-medium transition-
colors
            ${selected === lang.code
              ? 'bg-primary text-white'
              : 'bg-gray-100 text-gray-600 hover:bg-gray-200'
            }`}>
          {lang.label}
        </button>
      ))}
    </div>
  )
}
```

### LoadingSkeleton.tsx

Shown while waiting for the AI response. Pulsing grey blocks — not a spinner. Makes the wait feel like progress.

```
// src/components/LoadingSkeleton.tsx
export default function LoadingSkeleton() {
  return (
    <div className='space-y-4 animate-pulse'>
      <div className='h-8 bg-gray-200 rounded-xl w-1/3' />
      <div className='h-24 bg-gray-200 rounded-xl' />
      <div className='h-24 bg-gray-200 rounded-xl' />
      <div className='h-16 bg-gray-200 rounded-xl w-2/3' />
    </div>
  )
}
```

### CounsellorCard.tsx

Shown when crisis_flag is true. A prominent card with KNUST Counselling Centre details and a clear CTA.

```
// src/components/CounsellorCard.tsx
export default function CounsellorCard() {
```

```
  return (
    <div className='bg-wellness-bg border-2 border-wellness rounded-2xl p-6'>
      <div className='flex items-center gap-3 mb-3'>
        <span className='text-2xl'>💜</span>
        <h3 className='text-lg font-bold text-dark'>You don't have to face this
alone</h3>
      </div>
      <p className='text-base text-dark mb-4'>
        The KNUST Counselling Centre is here for you.
        Real people, free and confidential.
      </p>
      <div className='bg-white rounded-xl p-4 mb-4 space-y-1'>
        <p className='text-sm font-semibold text-dark'>KNUST Counselling
Centre</p>
        <p className='text-sm text-gray-600'>Near the University Hospital</p>
        <p className='text-sm text-gray-600'>Monday – Friday, 8am – 5pm</p>
      </div>
      <button className='w-full bg-wellness text-white py-3 rounded-xl
        font-semibold text-base hover:bg-purple-700 transition-colors'>
        Get Support Now
      </button>
    </div>
  )
}
```

## 04 Screen: Login & Sign Up

Route: /login — This is the first screen unauthenticated users see. Keep it minimal. The emotional register is calm and welcoming, not corporate.

### What this screen contains

WellCheck GH logo + tagline: 'Your daily wellness companion'
Tab toggle: Log In / Sign Up
Email input field
Password input field
Primary CTA button: 'Log In' or 'Create Account'
Subtle error message area (below the button, not a modal)

### Key UX rules for this screen:

- Max width 400px, centred on screen. Don't let the form stretch to full width.
- Background is surface (#F9FAFB) — no image, no gradient, just calm.
- On error (wrong password, email exists): show a small inline message in text-red-500 below the button. No alert() popup — ever.
- On success: redirect immediately to /checkin. No 'welcome' interstitial.
- Password field must have a show/hide toggle (eye icon from lucide-react).

### Full code — src/app/login/page.tsx:

```
'use client'
import { useState } from 'react'
import { useRouter } from 'next/navigation'
import { supabase } from '@/lib/supabase'
import { Eye, EyeOff } from 'lucide-react'

export default function LoginPage() {
  const router = useRouter()
  const [mode, setMode] = useState<'login' | 'signup'>('login')
  const [email, setEmail] = useState('')
  const [password, setPassword] = useState('')
  const [showPw, setShowPw] = useState(false)
  const [error, setError] = useState('')
  const [loading, setLoading] = useState(false)

  async function handleSubmit() {
    setError('')
    setLoading(true)
    try {
      if (mode === 'signup') {
```

```
        const { error } = await supabase.auth.signUp({ email, password })
        if (error) throw error
      } else {
        const { error } = await supabase.auth.signInWithPassword({ email, password
})
        if (error) throw error
      }
      router.push('/checkin')
    } catch (err: any) {
      setError(err.message || 'Something went wrong. Try again.')
    } finally {
      setLoading(false)
    }
  }

  return (
    <main className='min-h-screen bg-surface flex items-center justify-center p-
6'>
      <div className='w-full max-w-sm space-y-6'>
        <div className='text-center'>
          <h1 className='text-3xl font-bold text-dark'>WellCheck GH</h1>
          <p className='text-gray-500 mt-1 text-sm'>Your daily wellness
companion</p>
        </div>
        {/* Tab toggle */}
        <div className='flex bg-gray-100 rounded-xl p-1'>
          {(['login','signup'] as const).map(m => (
            <button key={m} onClick={() => setMode(m)}
              className={`flex-1 py-2 rounded-lg text-sm font-medium transition-
colors
                ${mode === m ? 'bg-white text-dark shadow-sm' : 'text-gray-
500'}`}>
              {m === 'login' ? 'Log In' : 'Sign Up'}
            </button>
          ))}
        </div>
        {/* Fields */}
        <div className='space-y-3'>
          <input type='email' placeholder='Email address' value={email}
            onChange={e => setEmail(e.target.value)}
            className='w-full px-4 py-3 rounded-xl border border-gray-200 text-
base
              focus:outline-none focus:ring-2 focus:ring-primary bg-white' />
          <div className='relative'>
            <input type={showPw ? 'text' : 'password'} placeholder='Password'
              value={password} onChange={e => setPassword(e.target.value)}
              className='w-full px-4 py-3 rounded-xl border border-gray-200 text-
base
                focus:outline-none focus:ring-2 focus:ring-primary bg-white pr-12'
/>
            <button onClick={() => setShowPw(!showPw)}
              className='absolute right-4 top-1/2 -translate-y-1/2 text-gray-400'>
              {showPw ? <EyeOff size={18}/> : <Eye size={18}/>}
            </button>
          </div>
        </div>
```

```
        {error && <p className='text-sm text-red-500 text-center'>{error}</p>}
        <button onClick={handleSubmit} disabled={loading}
          className='w-full bg-primary text-white py-4 rounded-xl font-semibold
            text-base hover:bg-primary/90 transition-colors disabled:opacity-50'>
          {loading ? 'Please wait...' : mode === 'login' ? 'Log In' : 'Create
Account'}
        </button>
      </div>
    </main>
  )
}
```

## 05 Screen: Check-In (Home)

Route: /checkin — This is the most important screen. Everything the user experiences starts here. It must feel open, calm, and inviting — not like a form.

### What this screen contains

Top nav: WellCheck GH logo (left), user avatar / logout icon (right)
Greeting: 'Good morning, [name]' or just 'How are you feeling today?'
Language selector (English / Twi / Ga / Ewe) — pill toggles
Large textarea input: warm placeholder text
Mic button (appears to the right of the send button when non-English is selected)
Submit button: 'Check In' — full width, primary blue
Bottom nav: Home (active), History, (Profile if time allows)

### The textarea — get this right:

- Placeholder: 'How are you feeling right now? Describe anything — your body, your mind, whatever is on you today.'
- Minimum height 120px, grows with content (use resize-none + min-h class).
- Soft border (#E5E7EB), turns primary blue on focus (ring-2 ring-primary).
- No label above it — the placeholder is the label. Labels make it feel like a form.
- When non-English language is selected: show small note below input — 'You can also record in [language] using the mic button.'

### Full code — src/app/checkin/page.tsx:

```
'use client'
import { useState, useRef } from 'react'
import { useRouter } from 'next/navigation'
import { Send } from 'lucide-react'
import LanguageSelector from '@/components/LanguageSelector'
import MicButton from '@/components/MicButton'
import { submitCheckin, submitVoiceCheckin } from '@/lib/api'
import { useAuth } from '@/lib/useAuth'

export default function CheckinPage() {
  const router = useRouter()
  const { user } = useAuth()
  const [message, setMessage] = useState('')
  const [language, setLanguage] = useState('en')
  const [micState, setMicState] =
useState<'idle'|'recording'|'processing'>('idle')
  const [loading, setLoading] = useState(false)
  const mediaRef = useRef<MediaRecorder | null>(null)
```

```
  const chunksRef = useRef<Blob[]>([])

  async function handleTextSubmit() {
    if (!message.trim() || !user) return
    setLoading(true)
    try {
      const result = await submitCheckin({ userId: user.id, message, language })
      // Store result in sessionStorage for the result page to read
      sessionStorage.setItem('checkin_result', JSON.stringify(result))
      router.push('/result')
    } catch (err) {
      console.error(err)
    } finally {
      setLoading(false)
    }
  }

  async function handleMicPress() {
    if (micState === 'idle') {
      // Start recording
      const stream = await navigator.mediaDevices.getUserMedia({ audio: true })
      const recorder = new MediaRecorder(stream)
      chunksRef.current = []
      recorder.ondataavailable = e => chunksRef.current.push(e.data)
      recorder.onstop = async () => {
        setMicState('processing')
        const blob = new Blob(chunksRef.current, { type: 'audio/wav' })
        const result = await submitVoiceCheckin({ userId: user!.id, language,
audio: blob })
        sessionStorage.setItem('checkin_result', JSON.stringify(result))
        router.push('/result')
      }
      recorder.start()
      mediaRef.current = recorder
      setMicState('recording')
    } else if (micState === 'recording') {
      // Stop recording
      mediaRef.current?.stop()
    }
  }

  return (
    <main className='min-h-screen bg-surface flex flex-col max-w-lg mx-auto'>
      {/* Nav */}
      <header className='flex items-center justify-between px-6 py-4 border-b
border-gray-100'>
        <span className='text-lg font-bold text-dark'>WellCheck GH</span>
        <button className='text-gray-500 text-sm'>Log out</button>
      </header>
      {/* Content */}
      <div className='flex-1 px-6 py-8 space-y-6'>
        <div>
          <h2 className='text-2xl font-bold text-dark'>How are you feeling?</h2>
          <p className='text-gray-500 text-sm mt-1'>Be as honest as you need to
be.</p>
        </div>
```

```
        <LanguageSelector selected={language} onChange={setLanguage} />
        <textarea
          value={message}
          onChange={e => setMessage(e.target.value)}
          placeholder='How are you feeling right now? Describe anything — your
body, your mind, whatever is on you today.'
          className='w-full min-h-[140px] px-4 py-4 rounded-xl border border-gray-
200 text-base
            focus:outline-none focus:ring-2 focus:ring-primary bg-white resize-
none'
        />
        {language !== 'en' && (
          <p className='text-sm text-gray-500'>
            You can also record in {language === 'tw' ? 'Twi' : language === 'ga'
? 'Ga' : 'Ewe'} using the mic button.
          </p>
        )}
        <div className='flex items-center gap-3'>
          <button onClick={handleTextSubmit} disabled={!message.trim() || loading}
            className='flex-1 bg-primary text-white py-4 rounded-xl font-semibold
              text-base hover:bg-primary/90 transition-colors disabled:opacity-40
flex
              items-center justify-center gap-2'>
            <Send size={18} />
            {loading ? 'Checking in...' : 'Check In'}
          </button>
          {language !== 'en' && (
            <MicButton state={micState} onPress={handleMicPress} />
          )}
        </div>
      </div>
    </main>
  )
}
```

## 06 Screen: Conversational Follow-Up

Route: /result (intermediate state) — This screen appears when the AI returns type: 'clarify'. The AI is asking one follow-up question before giving a final result. This is new in v2 of the backend.

ⓘ How clarify works: If the user's message is vague (e.g. 'I don't feel well'), the AI returns { type: 'clarify', follow_up_question: 'Can you tell me more — is it more physical or emotional?' } instead of a final result. You show the AI's question, the user answers, and you send the full conversation history back for the final response.

### What this screen contains

AI's follow-up question displayed as a chat bubble
The user's original message shown above it (context)
A text input for the user's follow-up answer
A 'Send' button to continue
Small label: 'One quick question before your result'

### How to handle it in code:

In your result page, check the type field first. If it's 'clarify', show the follow-up UI instead of the result cards. When the user answers, call /checkin again with the conversation_history array included:

```
// In src/app/result/page.tsx

// After receiving the response from /checkin:
if (result.type === 'clarify') {
  // Show the follow-up question screen
  // Store conversation history in state:
  setConversationHistory([
    { role: 'user',      content: originalMessage },
    { role: 'assistant', content: result.follow_up_question }
  ])
  setFollowUpQuestion(result.follow_up_question)
  setShowFollowUp(true)
  return
}

// When user submits their follow-up answer:
async function handleFollowUpSubmit() {
  const updatedHistory = [
    ...conversationHistory,
    { role: 'user', content: followUpAnswer }
  ]
  const finalResult = await submitCheckin({
    userId: user.id,
```

```
    message: followUpAnswer,
    language,
    conversation_history: updatedHistory  // Pass the full history
  })
  // Now show the real result
  setResult(finalResult)
  setShowFollowUp(false)
}
```

### The follow-up UI:

```
// Follow-up question UI (shown inside result page when showFollowUp is true)
{showFollowUp && (
  <div className='min-h-screen bg-surface flex flex-col max-w-lg mx-auto px-6 py-
8'>
    <p className='text-sm text-primary font-medium mb-6'>
      One quick question before your result
    </p>
    {/* AI question bubble */}
    <div className='bg-white rounded-2xl rounded-tl-sm p-4 shadow-sm border
border-gray-100 mb-6'>
      <p className='text-base text-dark'>{followUpQuestion}</p>
    </div>
    {/* User reply input */}
    <textarea
      value={followUpAnswer}
      onChange={e => setFollowUpAnswer(e.target.value)}
      placeholder='Type your answer here...'
      className='w-full min-h-[100px] px-4 py-3 rounded-xl border border-gray-200
        focus:outline-none focus:ring-2 focus:ring-primary bg-white resize-none
mb-4'
    />
    <button onClick={handleFollowUpSubmit}
      className='w-full bg-primary text-white py-4 rounded-xl font-semibold'>
      Continue
    </button>
  </div>
)}
```

## 07 Screen: Result — Normal

Route: /result — Shown after a successful check-in when crisis_flag is false and urgency is LOW or MEDIUM. The most common screen users will see.

### What this screen contains

Back button (top left) — returns to /checkin
Section header: 'Here is what we found'
UrgencyBadge component (if type is physical or both)
ResultCard: Possible Causes (if type is physical or both)
ResultCard: What To Do (action_advice)
ResultCard: How You Are Doing (wellness_response + coping_tip) — purple tinted
Button: 'Check In Again' — ghost style, bottom of page

### Conditional rendering rules — only show what applies:

| Condition | What to show |
|---|---|
| type === 'physical' | Urgency badge + Causes card + Action card only. Hide wellness card. |
| type === 'mental' | Wellness card + Coping tip only. Hide urgency badge, causes, action. |
| type === 'both' | Show everything: urgency + causes + action + wellness + coping. |
| crisis_flag === true | Skip this screen entirely — go to Red Alert screen instead. |
| urgency === null | Do not render the UrgencyBadge at all. |
| causes is empty array | Do not render the Causes card. |

### Full code — src/app/result/page.tsx (normal result section):

```
'use client'
import { useEffect, useState } from 'react'
import { useRouter } from 'next/navigation'
import UrgencyBadge from '@/components/UrgencyBadge'
import ResultCard from '@/components/ResultCard'
import CounsellorCard from '@/components/CounsellorCard'
import LoadingSkeleton from '@/components/LoadingSkeleton'

export default function ResultPage() {
  const router = useRouter()
  const [result, setResult] = useState<any>(null)
```

```
  useEffect(() => {
    const stored = sessionStorage.getItem('checkin_result')
    if (!stored) { router.push('/checkin'); return }
    const parsed = JSON.parse(stored)
    setResult(parsed.result)
  }, [])

  if (!result) return <div className='p-6'><LoadingSkeleton /></div>

  // Crisis → full-screen red alert
  if (result.crisis_flag) {
    return (
      <main className='min-h-screen bg-urgency-high-bg flex flex-col items-center
        justify-center p-6 max-w-lg mx-auto text-center'>
        <span className='text-6xl mb-6'>⚠</span>
        <h1 className='text-3xl font-bold text-urgency-high mb-4'>
          This needs immediate attention
        </h1>
        <p className='text-base text-dark mb-8'>{result.wellness_response}</p>
        <CounsellorCard />
        <button onClick={() => router.push('/checkin')}
          className='mt-6 text-sm text-gray-500 underline'>
          Back to home
        </button>
      </main>
    )
  }

  return (
    <main className='min-h-screen bg-surface max-w-lg mx-auto px-6 py-8 space-y-
4'>
      <button onClick={() => router.push('/checkin')}
        className='text-sm text-primary font-medium'>← Check in again</button>
      <h2 className='text-2xl font-bold text-dark'>Here is what we found</h2>

      {result.urgency && <UrgencyBadge level={result.urgency} />}

      {result.causes?.length > 0 && (
        <ResultCard title='Possible causes' icon='🔍' bgClass='bg-primary-light'>
          <ul className='space-y-1'>
            {result.causes.map((c: string) => (
              <li key={c} className='text-base text-dark'>• {c}</li>
            ))}
          </ul>
        </ResultCard>
      )}

      {result.action_advice && (
        <ResultCard title='What to do' icon='☑' bgClass='bg-white'>
          <p className='text-base text-dark'>{result.action_advice}</p>
        </ResultCard>
      )}
```

```
      {result.wellness_response && (
        <ResultCard title='How you are doing' icon='💗' bgClass='bg-wellness-bg'>
          <p className='text-base text-dark mb-3'>{result.wellness_response}</p>
          {result.coping_tip && (
            <div className='bg-white rounded-xl p-3 mt-2'>
              <p className='text-sm font-semibold text-dark mb-1'>Try this:</p>
              <p className='text-sm text-dark'>{result.coping_tip}</p>
            </div>
          )}
        </ResultCard>
      )}

      <button onClick={() => router.push('/checkin')}
        className='w-full border border-gray-200 text-dark py-3 rounded-xl text-sm
font-medium mt-4'>
        Check in again
      </button>
    </main>
  )
}
```

## 08 Screen: Result — Red Alert (HIGH Urgency)

This screen appears when urgency is HIGH or crisis_flag is true. It is already handled inside the result page code above (Section 07). This section covers the design rules for it specifically.

⚠ This is your biggest demo moment. When judges see the screen change completely to red and a bold warning appears, that is the reaction you want. Get this exactly right.

### Design rules — non-negotiable:

- The entire background turns #FEF2F2 (urgency-high-bg). Not just a card — the whole screen.
- The ⚠ icon is large (text-6xl = 60px+) and centered at the top.
- The heading is text-3xl font-bold in #DC2626 (urgency-high).
- ALL navigation, history, other cards are hidden. Nothing on screen except the alert and the counsellor card.
- No animation on the alert itself — it appears instantly and seriously. Animation here would undercut the gravity.
- The only interactive elements are the CounsellorCard CTA and a small 'Back to home' text link below.

### Two cases that trigger this screen:

| Case | What it looks like |
|---|---|
| urgency === 'HIGH' + crisis_flag false | Red alert + action_advice ('Go to hospital immediately') + CounsellorCard |
| crisis_flag === true (any urgency) | Red alert + wellness_response (warm redirect to counsellor) + CounsellorCard only. No medical causes shown. |

ⓘ The code for this is already inside the result page (Section 07). The if (result.crisis_flag) block handles crisis. For HIGH urgency without crisis, add a second check: if (result.urgency === 'HIGH') render the red background version with the action_advice and CounsellorCard.

## 09 Screen: Mood History

Route: /history — Shows the user's last 7 check-ins as a simple chart and a list. The purpose is to help students notice patterns — 'I've had 3 MEDIUM days this week.'

### What this screen contains

Screen title: 'Your Check-In History'

A simple bar or line chart (recharts) — x-axis is date, y-axis is urgency level (LOW=1, MEDIUM=2, HIGH=3)

A scrollable list below the chart — each item shows: date, type icon, urgency badge

Empty state (when no history): illustration placeholder + 'No check-ins yet. How are you feeling today?' + CTA button

Bottom nav with History tab active

### Full code — src/app/history/page.tsx:

```
'use client'
import { useEffect, useState } from 'react'
import { BarChart, Bar, XAxis, YAxis, Tooltip, ResponsiveContainer } from
'recharts'
import { getHistory } from '@/lib/api'
import { useAuth } from '@/lib/useAuth'
import UrgencyBadge from '@/components/UrgencyBadge'

const urgencyScore = { LOW: 1, MEDIUM: 2, HIGH: 3 }
const urgencyColor = { LOW: '#16A34A', MEDIUM: '#D97706', HIGH: '#DC2626' }

export default function HistoryPage() {
  const { user } = useAuth()
  const [history, setHistory] = useState<any[]>([])
  const [loading, setLoading] = useState(true)

  useEffect(() => {
    if (!user) return
    getHistory(user.id).then(data => {
      setHistory(data.history || [])
      setLoading(false)
    })
  }, [user])

  const chartData = history.map(item => ({
    date: new Date(item.created_at).toLocaleDateString('en-GH', { weekday: 'short'
}),
    score: urgencyScore[item.urgency as keyof typeof urgencyScore] || 0,
    color: urgencyColor[item.urgency as keyof typeof urgencyColor] || '#6B7280',
    urgency: item.urgency,
  })).reverse()
```

```
  if (loading) return <div className='p-6 text-gray-400'>Loading...</div>

  return (
    <main className='min-h-screen bg-surface max-w-lg mx-auto px-6 py-8'>
      <h2 className='text-2xl font-bold text-dark mb-6'>Your Check-In History</h2>

      {history.length === 0 ? (
        <div className='text-center py-20 space-y-4'>
          <p className='text-5xl'>🌱</p>
          <p className='text-gray-500 text-base'>No check-ins yet.</p>
          <a href='/checkin' className='text-primary font-medium text-sm'>
            How are you feeling today? →
          </a>
        </div>
      ) : (
        <>
          {/* Chart */}
          <div className='bg-white rounded-2xl p-4 mb-6 border border-gray-100'>
            <ResponsiveContainer width='100%' height={160}>
              <BarChart data={chartData}>
                <XAxis dataKey='date' tick={{ fontSize: 12 }} />
                <YAxis domain={[0, 3]} hide />
                <Tooltip formatter={(v) => ['', '']}
                  content={({ active, payload }) => active && payload?.[0] ? (
                    <div className='bg-white p-2 border rounded shadow text-sm'>
                      {payload[0].payload.urgency || 'No urgency'}
                    </div>
                  ) : null} />
                <Bar dataKey='score' fill='#1A56DB' radius={[4,4,0,0]} />
              </BarChart>
            </ResponsiveContainer>
          </div>
          {/* List */}
          <div className='space-y-3'>
            {history.map(item => (
              <div key={item.id} className='bg-white rounded-xl p-4 border border-
gray-100
                flex items-center justify-between'>
                <div>
                  <p className='text-sm font-medium text-dark
capitalize'>{item.type}</p>
                  <p className='text-xs text-gray-400 mt-0.5'>
                    {new Date(item.created_at).toLocaleDateString('en-GH', {
                      day: 'numeric', month: 'short', hour: '2-digit', minute: '2-
digit'
                    })}
                  </p>
                </div>
                <UrgencyBadge level={item.urgency} />
              </div>
            ))}
          </div>
        </>
      )}
```

```
      </main>
    )
  }
```

## 10 Screen: Counsellor Admin View

Route: /admin — Desktop-only view. The counsellor logs in and sees all flagged (crisis) sessions. They can mark sessions as resolved. This is your 'system thinking' demo moment — showing judges you built for the institution, not just the student.

ⓘ For the hackathon, there's no separate counsellor login. The /admin route is accessible to anyone who is logged in. In a real product, you'd add a role check. For the demo, just navigate directly to /admin.

### What this screen contains

Header: 'Counsellor Dashboard' + subtitle: 'Flagged sessions requiring attention'

Stats row: Total Flagged / Unresolved / Resolved counts

Table: each row = one flagged session (date, user ID truncated, input preview, status badge, Resolve button)

On clicking Resolve: calls PATCH /api/flagged/resolve, row updates to 'Resolved' status

Empty state: 'No flagged sessions — all clear.'

### Full code — src/app/admin/page.tsx:

```
'use client'
import { useEffect, useState } from 'react'
import { getFlagged, resolveFlagged } from '@/lib/api'

export default function AdminPage() {
  const [sessions, setSessions] = useState<any[]>([])
  const [loading, setLoading] = useState(true)

  useEffect(() => {
    getFlagged().then(data => {
      setSessions(data.flagged || [])
      setLoading(false)
    })
  }, [])

  async function handleResolve(id: string) {
    await resolveFlagged(id)
    setSessions(prev => prev.map(s => s.id === id ? { ...s, resolved: true } : s))
  }

  const unresolved = sessions.filter(s => !s.resolved).length

  return (
    <main className='min-h-screen bg-surface p-8'>
      <div className='max-w-4xl mx-auto'>
```

```
        <h1 className='text-2xl font-bold text-dark mb-1'>Counsellor
Dashboard</h1>
        <p className='text-gray-500 text-sm mb-8'>Flagged sessions requiring
attention</p>

        {/* Stats */}
        <div className='grid grid-cols-3 gap-4 mb-8'>
          {[
            { label: 'Total Flagged', value: sessions.length, color: 'text-dark'
},
            { label: 'Unresolved',    value: unresolved,       color: 'text-
urgency-high' },
            { label: 'Resolved',      value: sessions.length - unresolved, color:
'text-urgency-low' },
          ].map(stat => (
            <div key={stat.label} className='bg-white rounded-xl p-4 border
border-gray-100'>
              <p className={`text-3xl font-bold ${stat.color}`}>{stat.value}</p>
              <p className='text-sm text-gray-500 mt-1'>{stat.label}</p>
            </div>
          ))}
        </div>

        {/* Table */}
        {sessions.length === 0 ? (
          <div className='text-center py-16 bg-white rounded-2xl border border-
gray-100'>
            <p className='text-2xl mb-2'>☑</p>
            <p className='text-gray-500'>No flagged sessions — all clear.</p>
          </div>
        ) : (
          <div className='bg-white rounded-2xl border border-gray-100 overflow-
hidden'>
            <table className='w-full'>
              <thead className='bg-gray-50 border-b border-gray-100'>
                <tr>
                  {['Date','User','Message Preview','Status','Action'].map(h => (
                    <th key={h} className='text-left px-4 py-3 text-xs font-
semibold
                      text-gray-500 uppercase tracking-wide'>{h}</th>
                  ))}
                </tr>
              </thead>
              <tbody className='divide-y divide-gray-50'>
                {sessions.map(s => (
                  <tr key={s.id} className='hover:bg-gray-50'>
                    <td className='px-4 py-3 text-sm text-gray-600'>
                      {new Date(s.created_at).toLocaleDateString('en-GH')}
                    </td>
                    <td className='px-4 py-3 text-sm font-mono text-gray-500'>
                      {s.user_id.slice(0,8)}...
                    </td>
                    <td className='px-4 py-3 text-sm text-dark max-w-xs truncate'>
                      {s.raw_input}
                    </td>
                    <td className='px-4 py-3'>
```

```
                      <span className={`text-xs font-bold px-2 py-1 rounded-full
                        ${s.resolved ? 'bg-urgency-low-bg text-urgency-low' : 'bg-
urgency-high-bg text-urgency-high'}`}>
                        {s.resolved ? 'Resolved' : 'Open'}
                      </span>
                    </td>
                    <td className='px-4 py-3'>
                      {!s.resolved && (
                        <button onClick={() => handleResolve(s.id)}
                          className='text-xs text-primary font-medium
hover:underline'>
                          Mark resolved
                        </button>
                      )}
                    </td>
                  </tr>
                ))}
              </tbody>
            </table>
          </div>
        )}
      </div>
    </main>
  )
}
```

## 11 API Integration — Calling Darktrace's Backend

All API calls live in src/lib/api.ts. This is the single file that talks to Darktrace's FastAPI server. Every page imports from here — never make fetch calls directly inside page components.

### Full code — src/lib/api.ts:

```
const BASE = process.env.NEXT_PUBLIC_API_BASE_URL || 'http://localhost:8000'

// ── /checkin ──────────────────────────────────────────────────
export async function submitCheckin({
  userId, message, language = 'en', conversation_history = []
}: {
  userId: string
  message: string
  language?: string
  conversation_history?: { role: string; content: string }[]
}) {
  const res = await fetch(`${BASE}/api/checkin`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ user_id: userId, message, language,
conversation_history })
  })
  if (!res.ok) throw new Error('Check-in failed')
  return res.json()
}

// ── /checkin-voice ────────────────────────────────────────────
export async function submitVoiceCheckin({
  userId, language, audio
}: { userId: string; language: string; audio: Blob }) {
  const form = new FormData()
  form.append('user_id', userId)
  form.append('language', language)
  form.append('audio', audio, 'recording.wav')
  const res = await fetch(`${BASE}/api/checkin-voice`, { method: 'POST', body:
form })
  if (!res.ok) throw new Error('Voice check-in failed')
  return res.json()
}

// ── /history ──────────────────────────────────────────────────
export async function getHistory(userId: string, limit = 7) {
  const res = await fetch(`${BASE}/api/history/${userId}?limit=${limit}`)
  if (!res.ok) throw new Error('Could not fetch history')
  return res.json()
}

// ── /flagged ──────────────────────────────────────────────────
export async function getFlagged() {
```

```
  const res = await fetch(`${BASE}/api/flagged`)
  if (!res.ok) throw new Error('Could not fetch flagged sessions')
  return res.json()
}

// ── /flagged/resolve ──────────────────────────────────────────
export async function resolveFlagged(sessionId: string) {
  const res = await fetch(`${BASE}/api/flagged/resolve`, {
    method: 'PATCH',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ session_id: sessionId })
  })
  if (!res.ok) throw new Error('Could not resolve session')
  return res.json()
}
```

## 12 Auth Flow — Supabase

Two files handle auth: the Supabase client setup and a useAuth hook that gives every page access to the current user.

### src/lib/supabase.ts — Supabase client:

```
import { createClient } from '@supabase/supabase-js'

export const supabase = createClient(
  process.env.NEXT_PUBLIC_SUPABASE_URL!,
  process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!
)
```

### src/lib/useAuth.ts — Auth hook:

```
'use client'
import { useEffect, useState } from 'react'
import { supabase } from './supabase'
import type { User } from '@supabase/supabase-js'

export function useAuth() {
  const [user, setUser] = useState<User | null>(null)
  const [loading, setLoading] = useState(true)

  useEffect(() => {
    // Get current session
    supabase.auth.getSession().then(({ data }) => {
      setUser(data.session?.user ?? null)
      setLoading(false)
    })
    // Listen for auth changes
    const { data: listener } = supabase.auth.onAuthStateChange((_event, session)
=> {
      setUser(session?.user ?? null)
    })
    return () => listener.subscription.unsubscribe()
  }, [])

  return { user, loading }
}
```

### Route protection — redirect if not logged in:

Add this to any page that requires auth (checkin, result, history, admin):

```
const { user, loading } = useAuth()
const router = useRouter()

useEffect(() => {
  if (!loading && !user) router.push('/login')
}, [user, loading])

if (loading || !user) return null  // Don't render until auth is confirmed
```

## 13 Language Selector & Audio Playback

When the user selects Twi, Ga, or Ewe, two things change: the mic button appears, and the API response includes an audio_response_base64 field that you play back automatically.

### Playing back the Khaya TTS audio response:

After a successful voice check-in, the result object contains audio_response_base64 — a base64-encoded .wav file spoken in the user's chosen language. Play it automatically when the result screen loads:

```
// In result page — play audio if it exists
useEffect(() => {
  const stored = sessionStorage.getItem('checkin_result')
  if (!stored) return
  const parsed = JSON.parse(stored)
  setResult(parsed.result)

  // Auto-play the spoken response if it exists
  if (parsed.audio_response_base64) {
    const binary = atob(parsed.audio_response_base64)
    const bytes = new Uint8Array(binary.length)
    for (let i = 0; i < binary.length; i++) bytes[i] = binary.charCodeAt(i)
    const blob = new Blob([bytes], { type: 'audio/wav' })
    const url = URL.createObjectURL(blob)
    const audio = new Audio(url)
    audio.play().catch(() => {}) // Graceful fail if autoplay is blocked
  }
}, [])
```

ⓘ Browsers block autoplay without user interaction. The audio may not play automatically on first load if the user hasn't interacted with the page. As a fallback, show a small 'Play response' button that triggers audio.play() on click. This also gives users with hearing difficulties an obvious control.

## 14 Deployment to Vercel

Vercel is made by the same team that makes Next.js. Deploying is almost automatic.

### Step 1 — Push to GitHub

```
git init
git add .
git commit -m 'initial frontend'
git branch -M main
git remote add origin https://github.com/your-username/wellcheck-web.git
git push -u origin main
```

⚠ Make sure .env.local is in .gitignore before pushing. Never push your API keys.

### Step 2 — Deploy on Vercel

1. Go to vercel.com and sign up with GitHub
2. New Project → Import → select wellcheck-web
3. In the Environment Variables section, add all three variables from your .env.local
4. Set NEXT_PUBLIC_API_BASE_URL to Darktrace's Railway URL (not localhost)
5. Click Deploy — Vercel builds and deploys in under 2 minutes

Vercel gives you a URL like: https://wellcheck-web.vercel.app

💡 Every push to the main branch auto-deploys. So if you fix a bug during the hackathon, just push to main and it's live in 90 seconds.

## 15 Demo Prep & Order of Work

### Order of work — do this in order, do not skip ahead:

| Priority | Task |
|---|---|
| 1 — Do first | Project setup, Tailwind config, folder structure (Section 01) |
| 2 | Build all reusable components (Section 03) — these unblock everything else |
| 3 | Login screen (Section 04) — you need auth to test other screens |
| 4 — Core flow | Check-In screen (Section 05) — connect to Darktrace's /checkin endpoint |
| 5 | Result screen — normal flow (Section 07) — this is your main demo screen |
| 6 | Red alert screen (Section 08) — it's inside the result page, just the conditional |
| 7 | Mood history screen (Section 09) |
| 8 | Counsellor admin view (Section 10) |
| 9 | Conversational follow-up (Section 06) — only after core flow works |
| 10 | Audio playback for voice response (Section 13) |
| 11 — Last | Deploy to Vercel (Section 14), test on real phone browser |

### Demo path to rehearse (do this twice as a team before 4pm):

6. Open the app on your laptop (web) and Nemesis's phone (mobile) side by side
7. Log in with a pre-created test account — don't sign up live, too risky
8. Type: 'I have a headache and I've been really stressed about finals' → submit
9. Show the result: MEDIUM urgency badge, causes card, wellness response
10. Go to History — show the chart with the entry logged
11. Now type: 'I have chest pain and I can't breathe' → submit → show RED ALERT
12. Show CounsellorCard appearing — say: 'the app steps back, it doesn't try to be a doctor'
13. Open /admin → show the flagged session from step 6 → click Resolve
14. If time: switch to Twi, record audio, show the transcription + spoken response

💡 Prep a second browser tab with /admin already open. Switching tabs mid-demo is faster and less risky than navigating there live.

**Build the components first. Get auth working. Wire up /checkin.**

Everything else follows from those three things. You've got this, Carey.